

Nombre : Angelo Aram

Apellido : Alama Quesada

Código: 222 00002

Fecha

Examen de Desarrollo de sistemas móviles - Nivel Básico - Inform

2.

I. Análisis de Arquitectura y Concurrencia
Valor: 2pts c/u

1. El patrón repositorio como SSOT

Es la Única fuente de verdad porque centraliza el acceso a los datos; la UI no busca en varios lados, solo le pregunta al repositorio. Es una buena idea porque oculta todo el desorden de atrás con métodos limpios

Como funciona: El repositorio expone un flow conectando a la BD local (Room). Cuando necesita actualizar este llama a la así con Retrofit en segundo plano y guarda el resultado en Room. Como Room es reactivo, la UI se actualiza sola sin saber de enterarse de donde vino el dato

2. Corrutinas y Structured Concurrency

Es el ámbito de corrutinas acoplado al ciclo de vida del View Model. Sirve para aplicar concurrencia estructurada que esto evita que tareas queden flotando si la pantalla muere.

Si se cierra la pantalla el View Model se destruye y cancela el View Model Scope en cascada. La petición de Retrofit en curso se cancela en el acto, cancellation exception, esto libera recursos y lo que hace es evitar bug de memoria.

Nombre: Aluma Quezada, Angelo Aaron

3: Flujos Reactivos (cold vs hot)

El flow este detmide osea no emite nada hasta que lo escuchan con el collect ademas no guarda datos.

El state flow siempre esta activo osea emite como sin oyentes y retiene el ultimo estado.

4: Inyección de dependencias (DI) Manual

Funciona como un contenedor global. Como vive toda la app, permite instanciar los objetos pasados como el room o retrofit una sola vez y compartirlas con los viewmodels.

el by lazy es como una inicializacion perezosa.

Osea no crea los objetos al arrancar la app, esto para que no se ponga lento, sino que lo hace en el milisegundo exacto en el que se necesita por primera vez, esto ahorra memoria.

3.-

II Sección de Completes

1.- El flujo de datos que emite automáticamente cada vez que ~~completa~~ la acción una tabla de Room cambia de implementa usando el tipo Flow

2.- El operador Dispatchers.Default permite ejecutar tareas pesadas fuera del main thread, es específicamente optimizada para tareas de CPU como el parseo masivo de datos.

3.- Para que un servicio en primer plano de ubicación sea válido en android 10+, debe declararse en el manifiesto con el tipo location en el atributo android:foregroundServiceType.

4.- La función onCreate de la clase Application es la que garantiza que los componentes compartan la misma instancia del repositorio mediante un "cast".

5.- El componente que permite transformar una api basada en callbacks antiguos en función suspendible moderna se llama suspendable cancellable coroutine.

6.- El archivo de metadatos que describe la estructura de la app al sistema operativo se llama exactamente AndroidManifest.xml.

7.- En la arquitectura AOSP, la capa que actúa como puente estándar entre el framework y los controladores se denomina HAL.

8.- El operador de flow que permite combinar múltiples fuentes (como GPS y sensores) para emitir un estado unificado es combine.

4.

III Desarrollo de código y lógica personalizada.

1.- Repositorio y DI

```
class UsuarioRepository (private val usuarioDao: UsuarioDao) {  
    // aquí va la lógica del repo  
}
```

```
class MyApplication : Application() {
```

```
    private val database by lazy {  
        AppDatabase.getInstance(this)
```

```
    }
```

```
    val usuarioRepository by lazy {
```

```
        UsuarioRepository(database.usuarioDao())
```

```
    }
```

```
}
```


2. Lógica Dinámica de Identidad

// a) data class con los atributos solicitados

```
data class Usuario(  
    val nombre: String?,  
    val apellidos: String?,  
    val edad: Int?  
)
```

fun main() {

// b) Objeto con datos reales

```
val usuario = Usuario(  
    nombre = "Angela Acram",  
    apellidos = "Alena Quesada",  
    edad = 25
```

// c) y el cálculo del puntaje de identidad asegurando

// con el operador Elvis si son nulos

```
val letrasNombre = usuario.nombre?.replace(" ", "")?.length
```

```
val letrasApellidos = usuario.apellidos?.replace(" ", "")?.length
```

```
val puntajeIdentidad = letrasNombre + letrasApellidos
```

// d) Impresión del mensaje final

```
val nombreCompleto = "${usuario.nombre} ${usuario.apellidos}".trim()
```

```
println("Usuario: $nombreCompleto, tu puntaje de identidad es: $puntajeIdentidad")
```

```
}
```


5.-

IV Arquitectura del AOSP completa

1. Gráfico

1. Capa de Aplicaciones (Apps)
2. Marco de trabajo - Framework
3. Bibliotecas Nativas
4. Capa de Abstracción de Hardware
5. Núcleo Linux

2. Identificación

a. CA: Es la capa superior con la que interactúa el usuario. Contiene las aplicaciones del sistema como las que podrían estar instaladas por otros, terceros.

b. Framework: Código Kotlin que expone los servicios del sistema a las apps.

c. Bibliotecas Nativas y ART: Código C/C++ pasado, junto al motor de ejecución. En ART están los core libraries de Java y también el compilador que traduce el byte code a código nativo optimizado (los hilos).

d. HAL: El puente que estandariza la comunicación entre el sistema y el hardware de cualquier fabricante.

e. Kernel Linux: Lo hace manejar directamente los drivers, los hilos de hardware y la memoria física.